

Brute Force - Searching & String Matching

Sequential Search, Brute Force String Matching

Guide : Dr. Sriramy
Name : J. Bharath

Sequential Search

Question 1

Write a program to implement Sequential Search to find a given element in an array.

Input:

Array: 12, 25, 8, 45, 32, 19, 50

Key:

32

Output:

Element found at position 5

Number of comparisons = 5

```
▶ a=list(map(int,input().split()))
  k=int(input())
  for i,x in enumerate(a):
      if x==k:
          print("Found at position",i+1)
          breaks
```

```
... 12 25 8 45 32 19 50
     32
     Found at position 5
```

Question 2

Implement Sequential Search for an unsuccessful search.

Input:

Array: 5, 10, 15, 20, 25, 30, 35

Key:

18

Output:

Element not found

Number of comparisons = 7

```
▶ a=list(map(int,input().split()))  
  k=int(input())  
  print("Element Found" if k in a else "Element Not Found")
```

```
... 5 10 15 20 25 30 35  
     18  
     Element Not Found
```

Question 3

Write a program to find the first occurrence of an element using Sequential Search.

Input:

Array: 10, 25, 15, 25, 30, 25,

Key:

25

Output:

First occurrence at position 2

```
▶ a=list(map(int,input().split()))  
  k=int(input())  
  print(a.index(k)+1)
```

```
... 10 25 15 25 30 25 40  
     25  
     2
```

Question 4

Implement Sequential Search to find all occurrences of a given element.

Input:

Array: 7, 12, 7, 25, 18, 7, 30, 7

Key: 7

Output:

Occurrences at positions:

1, 3, 6, 8

Total occurrences = 4

```
▶ a=list(map(int,input().split()))  
  k=int(input())  
  print([i+1 for i,x in enumerate(a) if x==k])
```

```
... 7 12 7 25 18 7 30 7  
     7  
     [1, 3, 6, 8]
```

Question 5

Write a program to count:

- Total comparisons
- Total matches
- Total

mismatches

during Sequential

Search.

Input:

Array: 3, 6, 9, 12, 15, 18, 21

Key: 15

```
▶ a=list(map(int,input().split()))
  k=int(input())
  c=0
  for x in a:
      c+=1
      if x==k:
          break
  print("Comparisons =",c)
```

```
... 3 6 9 12 15 18 21
     15
     Comparisons = 5
```

Question 6

Implement Sentinel Sequential Search and compare it with ordinary Sequential Search.

Input:

Array: 14, 9, 22, 35, 18, 41, 27

Key:

18

Output:

Position found

Comparison count

```
▶ a=list(map(int,input().split()))
  k=int(input())
  a.append(k)
  i=0
  while a[i]!=k:
      i+=1
  print(i+1 if i<len(a)-1 else "Not Found")
```

```
... 14 9 22 35 18 41 27
      18
      5
```

Question 7

Write a program to perform Sequential Search on an array of student register numbers.

Input:

Register Numbers:

101, 102, 103, 104, 105, 106

Search Key: 104

Output:

Register Number found at position 4

```
▶ a=list(map(int,input().split()))
  k=int(input())
  print(a.index(k)+1)
```

```
... 101 102 103 104 105 106
      104
      4
```

Question 8

Implement Sequential Search for string data.

Input:

Names:

Anu, Bala, Charan, Deepa, Esha, Farhan

Search Name:

Deepa

Output:

Name found at position 4

```
▶ a=input().split()
  k=input()
  print(a.index(k)+1)
```

```
*** Anu Bala Charan Deepa Esha Farhan
    Deepa
    4
```

Question 9

Write a program to perform Sequential Search on a 2-D matrix.

Input:

12 8 15

5 18 27

9 11 24

Key = 24

Output:

Element found at Row 3 Column 3

```
▶ m=[list(map(int,input().split())) for _ in range(3)]
  k=int(input())
  for i in range(3):
    if k in m[i]:
      print(i+1,m[i].index(k)+1)
```

```
... 12 8 15
     5 18 27
     9 11 24
     24
     3 3
```

Question 10

Implement Sequential Search and analyze its performance.

Input:

Array:

45, 23, 67, 12, 89, 34, 56, 78, 90, 11,

29, 73, 18, 64, 37

Search Keys:

73

18

100

Tasks:

1. Show each comparison.

2. Count comparisons for every search.
3. Identify best, average, and worst cases.
4. Determine:
 - Best-case complexity
 - Average-case complexity
 - Worst-case complexity
 - Space complexity

```
▶ a=list(map(int,input().split()))
  k=int(input())
  c=0
  for x in a:
      c+=1
      if x==k:
          break
  print("Comparisons =",c)
```

```
... 45 23 67 12 89 34 56 78 90 11 29 73 18 64 37
     73
     Comparisons = 12
```

Brute Force String Matching

Question 1

Implement the Brute Force String Matching Algorithm to search for a pattern in a given text.

Input:

- Text = "AABAACAADAABAABA"
- Pattern =

"AABA" Output:

- Position(s) where the pattern occurs.
- Total number of comparisons.

```
▶ t=input()
  p=input()
  for i in range(len(t)-len(p)+1):
      if t[i:i+len(p)]==p:
          print(i)
```

```
... AABAACAADAABAABA
    AABA
    0
    9
    12
```

Question 2

Write a program to find all occurrences of a pattern in a text using Brute Force String Matching.

Input:

- Text = "BANANABANANA"
- Pattern =

"ANA" Output:

- Occurrences at positions 1, 3, 7, 9.

```
▶ t=input()  
p=input()  
for i in range(len(t)-len(p)+1):  
    if t[i:i+len(p)]==p:  
        print(i)
```

```
*** BANANABANANA  
ANA  
1  
3  
7  
9
```

Question 3

Implement Brute Force String Matching and display the comparison table for each alignment.

Input:

- Text = "MISSISSIPPI"
- Pattern =

"ISSI" Output:

- Shift number
- Comparisons made
- Match/Mismatch result

```
t=input()
p=input()
for i in range(len(t)-len(p)+1):
    print(i,"Match" if t[i:i+len(p)]==p else "No Match")
```

```
... MISSISSIPPI
    ISSI
    0 No Match
    1 Match
    2 No Match
    3 No Match
    4 Match
    5 No Match
    6 No Match
    7 No Match
```

Question 4

Write a program to count:

- Total character comparisons
- Total matches
- Total mismatches

during Brute Force String Matching.

Input:

- Text = "ABABABABAB"
- Pattern = "ABAB"

```
t=input()
p=input()
m=0
for i in range(len(t)-len(p)+1):
    if t[i:i+len(p)]==p:
        m+=1
print(m)
```

```
... ABABABABAB
    ABAB
    4
```

Question 5

Implement Brute Force String Matching and determine whether the search is:

- Best Case
- Average Case
- Worst

Case Input:

- Text = "AAAAAAAAAAB"
- Pattern = "AAAAB"

Display the number of comparisons performed.

```
▶ t=input()  
p=input()  
print("Found" if p in t else "Not Found")
```

```
... AAAAAAAAAAB  
    AAAAB  
    Found
```

Question 6

Write a program that searches for a pattern in a text and reports the first occurrence only using Brute Force String Matching.

Input:

- Text = "COMPUTERSCIENCE"
- Pattern =

"SCI" Output:

- First occurrence position
- Number of comparisons

```

▶ t=input()
  p=input()
  print(t.find(p))

... COMPUTERSCIENCE
   SCI
   8

```

Question 7

Implement Brute Force String Matching for case-insensitive searching.

Input:

- Text = "DataStructuresAndAlgorithms"
- Pattern =

"ALGORITHMS" Output:

- Pattern found position

```

▶ t=input().lower()
  p=input().lower()
  print(t.find(p))

... DataStructuresAndAlgorithms
   ALGORITHMS
   17

```

Question 8

Write a program to compare the execution of Brute Force String Matching for:

1. Successful search
2. Unsuccessful search

Input:

- Text = "PROGRAMMINGLAB"
- Pattern 1 = "LAB"
- Pattern 2 = "TEST"

Display the comparison count for both cases.

```
▶ t=input()  
p=input()  
print(t.find(p))
```

```
... PROGRAMMINGLAB  
LAB  
11
```

Question 9

Implement Brute Force String Matching and display every alignment of the pattern with the text.

Input:

- Text = "ABCDABCABCD"
- Pattern =

"ABCD" Output:

- Alignment number
- Matching result

- Pattern occurrence positions

```

t=input()
p=input()
for i in range(len(t)-len(p)+1):
    print(i,t[i:i+len(p)])

```

```

... ABCDABCABCD
    ABCDA
    0 ABCDA
    1 BCDAB
    2 CDABC
    3 DABCA
    4 ABCAB
    5 BCABC
    6 CABCD
    7 ABCDA

```

Question 10

Write a complete program for Brute Force String Matching and perform complexity analysis.

Input:

- Text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
- Pattern =

"TATCTT" Tasks:

1. Find all occurrences of the pattern.
2. Count total comparisons.
3. Display the shifting process.
4. Analyze:
 - Best-case complexity
 - Worst-case complexity
 - Space complexity

```

t=input()
p=input()
for i in range(len(t)-len(p)+1):
    if t[i:i+len(p)]==p:
        print(i)

```

```

... TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT
    TATCTT

```